

KUGGLE 1차 프로젝트

에어비앤비 호스트 및 소비자를 위한 숙소 적정 가격 예측

목차

01	주제선정 이유	03
02	분석개요	04
03	EDA 및 전처리	05
04	모델 및 TEST 데이터 예측	16
05	결론	22

주제선정 이유

에어비앤비 호스트 및 소비자를 위한 숙소 적정 가격 예측

왜 적정 가격 예측인가?

- 신규 호스트의 Pain-Point
- 가격 설정의 딜레마

프로젝트 목표

- 데이터 기반 객관적인 가격 예측 가이드 제공
 - > 신규 호스트의 성공적인 시장 안착 지원
 - > 소비자의 합리적인 소비 지원

사용데이터

New York City Airbnb Open Data

Airbnb listings and metrics in NYC, NY, USA (2019)
(<https://www.kaggle.com/datasets/dgomonov/new-york-city-airbnb-open-data>)

Columns

- price
- neighbourhood_group
- room_type
- latitude
- longitude
- availability_365

...

데이터 정제

- 불필요한 속성 제거
- 결측치 0으로 대치

의미 없어보이는 속성 제거

```
airbnb.drop(["id", "host_name", "last_review"], axis=1, inplace=True)
```

reviews_per_month 속성에 결측치 모두 0으로 채우기 및 확인

```
airbnb.fillna({"reviews_per_month":0}, inplace=True)
```

```
airbnb['reviews_per_month'].isnull().sum()
```

EDA 및 전처리

파생변수 생성

#뉴욕 5개 자치구별 중심점 좌표

```
centers = {
    'Manhattan': (40.7579747, -73.9855426),
    'Brooklyn': (40.6960679, -73.9845407),
    'Queens': (40.7595153, -73.8301274),
    'Bronx': (40.8179194, -73.9171183),
    'Staten Island': (40.6427017, -74.0799469)
}

center_lats = {k: v[0] for k, v in centers.items()}
center_lons = {k: v[1] for k, v in centers.items()}

airbnb['center_lat'] = airbnb['neighbourhood_group'].map(center_lats)
airbnb['center_lon'] = airbnb['neighbourhood_group'].map(center_lons)
```

-> 뉴욕 5개 자치구별 중심점 좌표

```
def haversine(lat1, lon1, lat2, lon2):
    R = 6371 # Earth radius in kilometers

    lat1_rad = np.radians(lat1)
    lon1_rad = np.radians(lon1)
    lat2_rad = np.radians(lat2)
    lon2_rad = np.radians(lon2)

    dlon = lon2_rad - lon1_rad
    dlat = lat2_rad - lat1_rad

    a = np.sin(dlat / 2)**2 + np.cos(lat1_rad) * np.cos(lat2_rad) * np.sin(dlon / 2)**2
    c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
    distance = R * c
    return distance
```

-> 하버사인 함수 - 위도와 경도로 두 지점 사이의 거리 계산

EDA 및 전처리

파생변수 생성

- 하버사인 함수 사용해서 dist_to_center 변수 생성

```
#하버사인 함수 적용
airbnb['dist_to_center'] = haversine(
    airbnb['latitude'], airbnb['longitude'],
    airbnb['center_lat'], airbnb['center_lon']
)
```

- Location Score

```
#해당 지역 중심점 기준 위치 점수(Location Score) 계산
airbnb['location_score'] = np.exp(-airbnb['dist_to_center'])
```

EDA 및 전처리

파생변수 생성

	name	neighbourhood_group	dist_to_center	location_score
0	Clean & quiet apt home by the park	Brooklyn	5.498274	0.004094
1	Skylit Midtown Castle	Manhattan	0.506717	0.602470
2	THE VILLAGE OF HARLEM....NEW YORK !	Manhattan	6.761551	0.001157
3	Cozy Entire Floor of Brownstone	Brooklyn	2.416984	0.089190
4	Entire Apt: Spacious Studio/Loft by central park	Manhattan	5.705918	0.003326

EDA 및 전처리

텍스트 마이닝

- 교통 / 방 상태 / 크기 관련 수식어가 이름에 들어간 숙소는 1, 없거나 결측치는 0으로 출력

```
transport_words = ['airport', 'subway', 'metro', 'station', 'transport',  
                  'location', 'shuttle', 'terminal', 'transit', 'bus']  
condition_words = ['lovely', 'cozy', 'cute', 'wonderful', 'beautiful',  
                  'charming', 'special', 'nice', 'bright', 'amazing',  
                  'gorgeous', 'unique', 'great', 'comfortable', 'luxury',  
                  'perfect', 'clean']  
big_size_words = ['large', 'spacious', 'huge', 'wide', 'roomy']  
  
def has_words(text, word_list):  
    if pd.isna(text):  
        return 0  
    text = text.lower()  
    return int(any(word in text for word in word_list))  
  
airbnb['is_transport_good'] = airbnb['name'].apply(lambda x: has_words(x, transport_words))  
airbnb['is_condition_good'] = airbnb['name'].apply(lambda x: has_words(x, condition_words))  
airbnb['is_size_good'] = airbnb['name'].apply(lambda x: has_words(x, big_size_words))
```

EDA 및 전처리

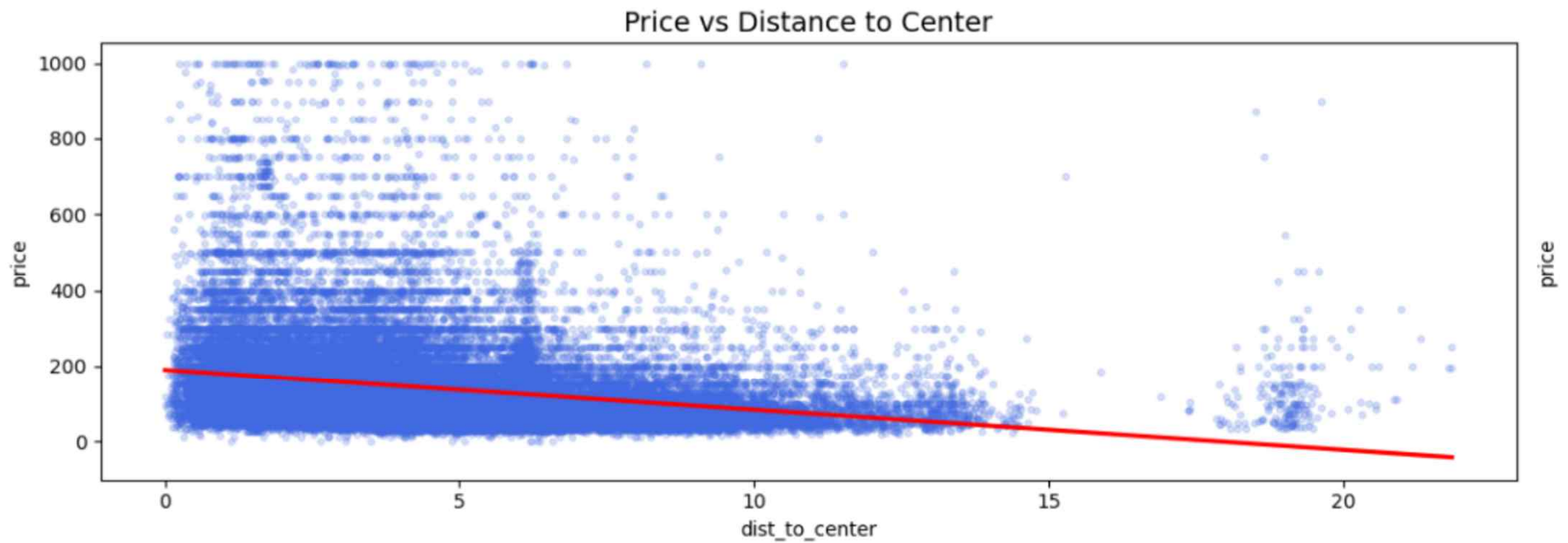
상관분석 산점도



-> 가격별 리뷰 수

EDA 및 전처리

상관분석 산점도

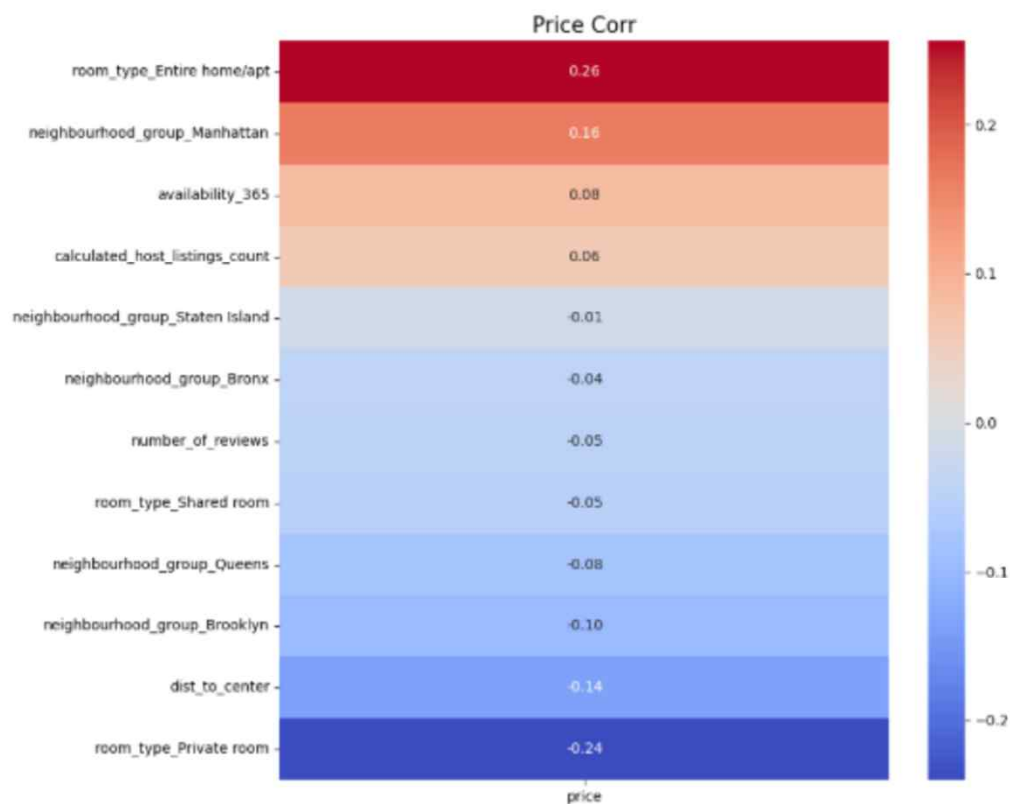


-> 가격별 중심과의 거리

EDA 및 전처리

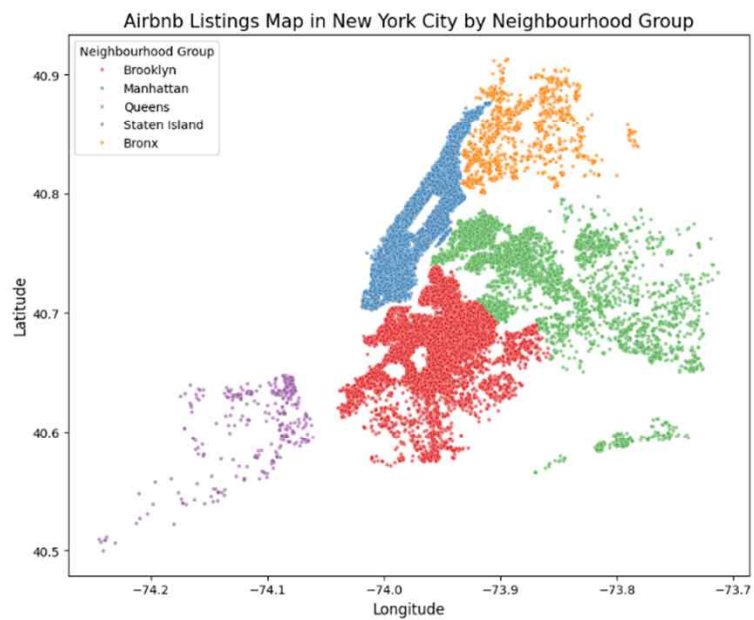
히트맵

- 가격 결정 요인 시각화
- 다른 요인보다
room type (객실 유형),
neighbourhood group (지역구)
에 따라 가격차이가 큼

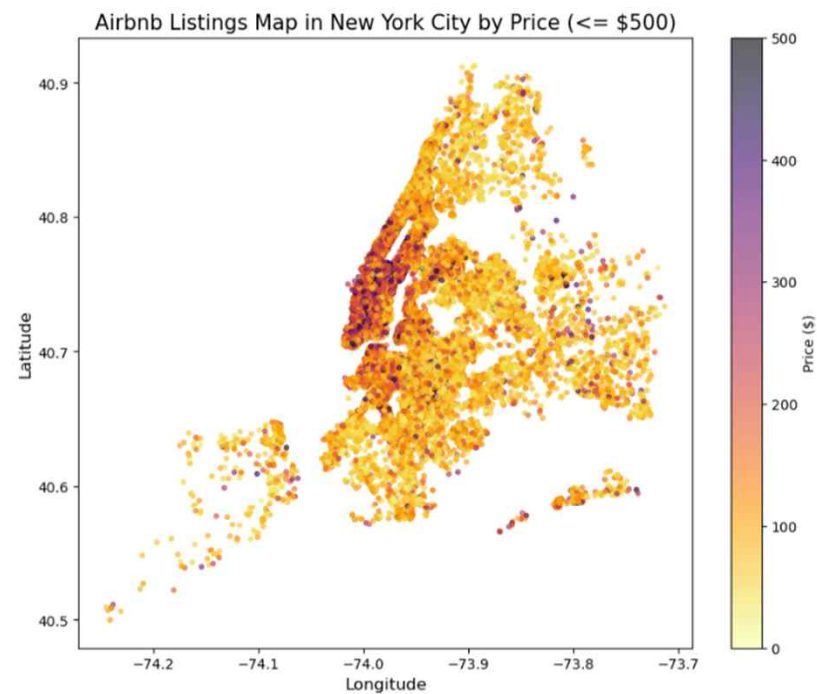


EDA 및 전처리

산점도



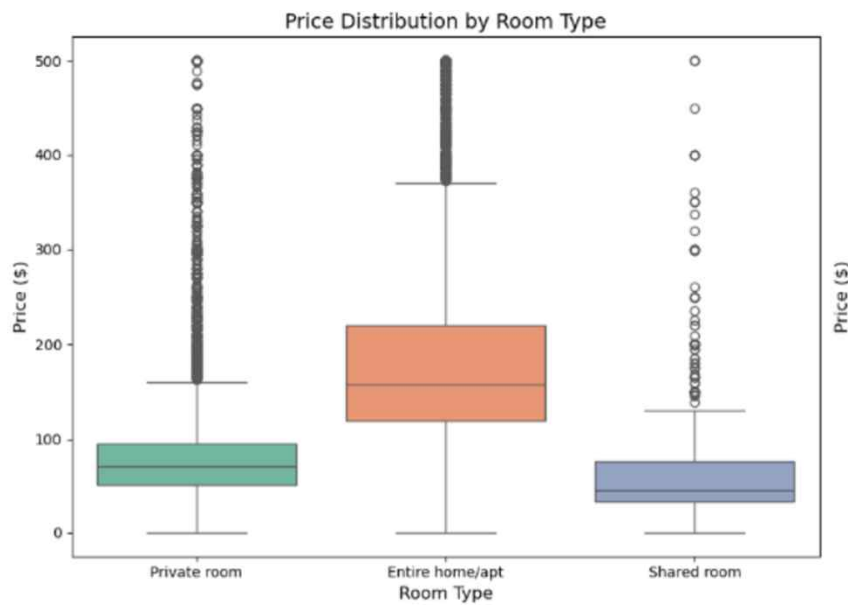
-> 지역구별 숙소 맵핑



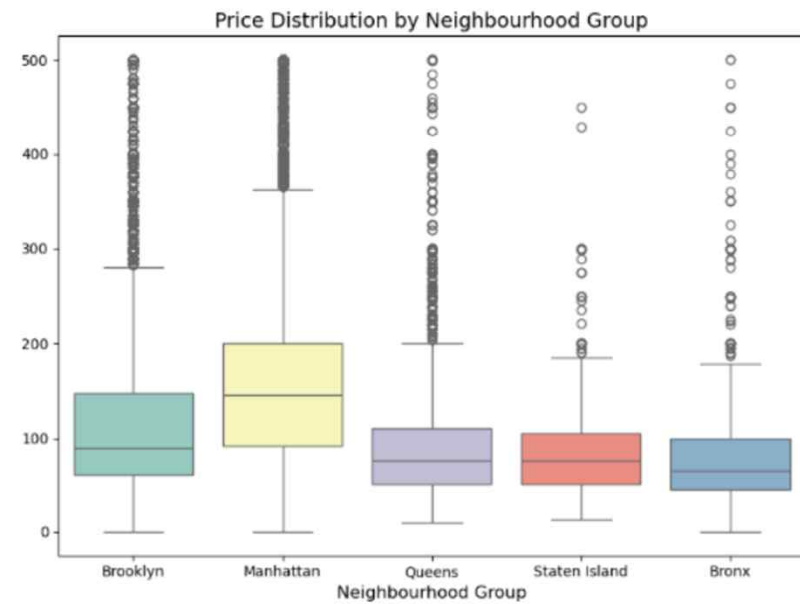
-> 가격별 숙소 맵핑

EDA 및 전처리

박스플롯



-> 객실 유형별 가격 분포



-> 지역구별 가격 분포

EDA 및 전처리

이상치 제거 및 로그변환

- 튀는 값 방지 위한 이상치 제거
- 왜곡 및 음수값 방지 위한 로그 변환

```
# 상위 5% 제외
cond = airbnb['price'] <= airbnb['price'].quantile(0.95)
airbnb_filtered = airbnb[cond]

# price 로그 변환
airbnb_filtered['price'] = np.log1p(airbnb_filtered['price'])
```

모델 및 TEST 데이터 예측

모델링 및 학습

- CatBoostRegressor
 - 지역구, 객실 유형 범주형 데이터 자동처리 가능
 - 빠른 예측 속도
- 학습용(Train) : 평가용(Test) = 8:2 분할

```
# 카테고리형 변수를 자동으로 처리하는 CatBoostRegressor 채택
# CatBoost 모델이 범주형 데이터를 인식할 수 있도록 컬럼명 지정
cat_features = ['neighbourhood_group', 'neighbourhood', 'room_type']

# 2. Train / Test 데이터 분할
# 학습용(Train) : 평가용(Test) = 8:2 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"학습용 데이터(Train) 크기: {X_train.shape}")
print(f"평가용 데이터(Test) 크기: {X_test.shape}")
```

학습용 데이터(Train) 크기: (37163, 16)
평가용 데이터(Test) 크기: (9291, 16)

모델 및 TEST 데이터 예측

모델링 및 학습

- CatBoost 모델을 활용해 1,000번의 반복 학습을 수행
- 오차가 줄어들지 않으면 중간에 멈추는 조기 종료 기능을 사용
- 지수 변환($\exp m1$)을 통해 우리가 실제로 이해할 수 있는 원래의 숙소 가격 단위로 복원

```
# 3. 머신러닝 모델 정의 및 학습
#CatBoost 모델링
model = CatBoostRegressor(
    iterations=1000,      # 트리를 최대 1000개 만들
    learning_rate=0.1,    # 학습률
    depth=6,              # 트리의 깊이
    cat_features=cat_features, # 범주형 변수 지정
    random_seed=42,
    verbose=100,          # 100번 학습할 때마다 진행 상황 출력
)

# 모델 학습 진행 (eval_set을 지정하면 오차가 더 이상 줄지 않을 때 조기 종료합니다)
print("\n[모델 학습을 시작합니다...]")
model.fit(
    X_train, y_train,
    eval_set=(X_test, y_test),
    early_stopping_rounds=50
)

# 4. 예측 및 성능 평가
y_pred=model.predict(X_test)
y_pred=np.expm1(y_pred)
y_test=np.expm1(y_test)
```

모델 및 TEST 데이터 예측

모델 평가

- 1,000번의 반복 학습 수행
- 과적합 방지 위해 오차가 감소하지 않을 경우 중간에 멈추는 조기 종료 기능 사용
- 지수 변환(expm1)을 통해 우리가 실제로 이해할 수 있는 원래의 숙소 가격 단위로 복원

평가 지표 계산

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)
```

```
print("=== 최종 모델 평가 결과 ===")
print(f"MAE (평균 절대 오차): {mae:.2f}")
print(f"RMSE (평균 제곱근 오차): {rmse:.2f}")
print(f"R2 (설명력): {r2:.4f}")
```

=== 최종 모델 평가 결과 ===

```
MAE (평균 절대 오차): 31.85
RMSE (평균 제곱근 오차): 47.23
R2 (설명력): 0.5741
```

모델 및 TEST 데이터 예측

모델 적용

```
# 1. 파일 저장
import pickle

with open('airbnb_rf_model.pkl', 'wb') as f: #write 권한 부여 + binary 형식
    pickle.dump(model, f) #파일 저장

def predict_newdata(input_data):
    # 모델 불러오기
    with open('airbnb_rf_model.pkl', 'rb') as f: #read 권한 부여 + binary 형식
        loaded_model = pickle.load(f)

    # location_score 전처리
    new_city = input_data['neighbourhood_group']
    new_c_lat, new_c_lon = centers[new_city]
    new_dist = haversine(input_data['latitude'], input_data['longitude'], new_c_lat, new_c_lon)
    new_loc_score = np.exp(-new_dist)
```

-> 모델 불러와 predict_newdata라는 새로운 함수 정의

```
# 입력데이터
input_df = pd.DataFrame(columns=X.columns) #X와 똑같은 컬럼을 가진 dataframe 생성
input_df.loc[0] = 0 #모든 컬럼 0으로 초기화

# 수치형 값 채우기
input_df['location_score'] = new_loc_score
input_df['availability_365'] = input_data['availability_365']
input_df['minimum_nights'] = input_data['minimum_nights']
input_df['number_of_reviews'] = input_data['number_of_reviews']
input_df['is_condition_good'] = 1 if input_data['is_condition_good'] else 0
input_df['is_transport_good'] = 1 if input_data['is_transport_good'] else 0
input_df['is_size_good'] = 1 if input_data['is_size_good'] else 0

# [4] 가격 예측
pred_price = loaded_model.predict(input_df)
pred_price = np.expm1(pred_price) # 로그 복원

return pred_price[0]
```

-> 입력받은 데이터를 모델이 계산할 수 있도록 DataFrame 형태로 가공

모델 및 TEST 데이터 예측

모델 적용

- 터미널을 통해 사용자에게 값을 입력받아 sample이라는 하나의 딕셔너리 생성
- sample 데이터를 predict_newdata()에 넣어 가격 예측

```
#사용자 입력
sample = {
    'latitude': float(input("위도 : ")),
    'longitude': float(input("경도 : ")),
    'neighbourhood_group': input("자치구 (Manhattan, Brooklyn, Queens, Bronx, Staten Island) : "),
    'room_type': input("4. 숙소 유형을 입력하세요 (Entire home/apt, Private room, Shared room) : "),
    'availability_365': int(input("1년 중 예약 가능 일수(0~365) : ")),
    'minimum_nights': int(input("최소 숙박 일수 : ")),
    'number_of_reviews': int(input("리뷰 개수 : ")),
    #y/n으로 입력받아 True/False로 변환 (입력하 y면 True)
    'is_condition_good': input("방 컨디션(좋음) 키워드 유무 (y/n): ").lower() == 'y',
    'is_transport_good': input("교통(좋음) 키워드 유무 (y/n): ").lower() == 'y',
    'is_size_good': input("방 사이즈(넓음) 키워드 유무 (y/n): ").lower() == 'y'
}

#예측 함수 호출
predicted_price = predict_newdata(sample)
print(f"예측 가격: {predicted_price:.2f}")
```

모델 및 TEST 데이터 예측

모델 적용

- 입력받은 데이터를 머신러닝 모델이 이해할 수 있는 형태로 데이터를 가공
- 가공을 마친 데이터를 미리 학습된 모델에 넣어 가격 수치를 계산

-> 임의로 입력한 값

```

위도 : 40.64749
경도 : -73.972
자치구 (Manhattan, Brooklyn, Queens, Bronx, Staten Island) : Brooklyn
4. 숙소 유형을 입력하세요 (Entire home/apt, Private room, Shared room) : Entire room
1년 중 예약 가능 일수(0~365) : 300
최소 숙박 일수 : 1
리뷰 개수 : 30
방 컨디션(좋음) 키워드 유무 (y/n): n
교통(좋음) 키워드 유무 (y/n): y
방 사이즈(넓음) 키워드 유무 (y/n): n
예측 가격: 88.95
  
```

↪ 예측한 가격

기대효과

호스트 측면

구축된 예측모델을 통해
자신의 위치와 조건에 맞는
객관적 기준가를 손쉽게 확인 가능

소비자 측면

예약하고자 하는 숙소의
조건을 모델에 입력
-> 호스트가 제시한 가격이
시장 시세 대비 적절한 수준인지
판단하는 보조 지표로 활용

감사합니다!